PROGRAMAÇÃO FUNCIONAL

MÔNADAS

CENTRO FEDERAL DE EDUCAÇÃO
TECNOLÓGICA DE MINAS GERAIS

MARCOS.EMILIANO@CEFETMG.BR
MARCOS EMILIANO

“If you're approaching Haskell monads a little nervously, that's **understandable**. You've probably heard that monads are a very powerful code-structuring technique, but … sometimes, with great power comes great (apparent) **nonsensicality**.”

PROBLEMA DO ENCADEAMENTO DE FUNÇÕES COM CONTEXTO

Em linguagens **imperativas**, a execução de um programa é **inerentemente sequencial**: executamos a Instrução A, depois a Instrução B, e o ponto e vírgula (;) atua como o cola-fluxo implícito.

PROBLEMA DO ENCADEAMENTO DE FUNÇÕES COM CONTEXTO

No entanto, em um ambiente funcional puro como o Haskell, não existem **instruções sequenciais** "gratuitas" — existem apenas **expressões** e **equações** matemáticas.

PROBLEMA DO ENCADEAMENTO DE FUNÇÕES COM CONTEXTO

Como **encadear** uma **sequência** de passos **dependentes** onde a **saída** de uma **etapa** serve de **entrada** para a **próxima**, especialmente quando cada etapa carrega um "**efeito colateral controlado**" (como falha, estado mutável ou E/S)?

PYRAMID OF DOOM

Considere a **busca encadeada** de dados em um **banco**: buscar um usuário, extrair seu perfil e obter sua preferência de tema. **Qualquer** uma dessas **etapas** pode retornar **Nothing**.

PYRAMID OF DOOM

```
-- Tipos de dados do domínio
type User = String
type Profile = String
type Theme = String

buscarUsuario :: Int -> Maybe User
buscarPerfil  :: User -> Maybe Profile
buscarTema    :: Profile -> Maybe Theme
```

PYRAMID OF DOOM

```
-- Abordagem Ingenuamente Manual (Sem
Abstração)
obterTemaUsuarioManual :: Int -> Maybe Theme
obterTemaUsuarioManual userId =
    case buscarUsuario userId of
        Nothing -> Nothing
        Just usr -> case buscarPerfil usr of
            Nothing -> Nothing
            Just prof -> case buscarTema prof of
                Nothing -> Nothing
                Just tema -> Just tema
```

PYRAMID OF DOOM

Verborragia e Poluição Sintática: O código fica profundamente aninhado para a direita.

Repetição de Lógica de Controle: A checagem do caso base Nothing -> Nothing é duplicada manualmente em todos os passos.

PYRAMID OF DOOM

Fragilidade: Adicionar um quarto ou quinto passo torna a função praticamente ininteligível.

Conclusão: Precisamos de um operador capaz de "engolir" essa checagem repetitiva de Nothing nos bastidores.

PROBLEMA DO ENCADEAMENTO DE FUNÇÕES COM CONTEXTO

Com Functores, aprendemos a aplicar **funções puras** (a -> b) a valores dentro de um **contexto** (f a -> f b).

PROBLEMA DO ENCADEAMENTO DE FUNÇÕES COM CONTEXTO

No entanto, na **prática**, muitas funções já **produzem** seus próprios **contextos**. Por exemplo, uma função de divisão segura não retorna um **Double**, mas sim um **Maybe Double**.

PROBLEMA DO ENCADEAMENTO DE FUNÇÕES COM CONTEXTO

Se tentarmos **encadear** múltiplas funções que geram **contextos** usando fmap, obteremos **contextos aninhados** indesejados (Maybe (Maybe Double) ou [[a]]).

PROBLEMA DO ENCADEAMENTO DE FUNÇÕES COM CONTEXTO

Como **encadear** sequencialmente **computações** que geram **contextos** sem **empilhar estruturas** infinitamente?

TYPECLASS MONAD

A classe de tipos **Monad** (presente no Prelude) é definida fundamentalmente por dois **métodos**:

```
1. class Monad m where
2.     return :: a -> m a
3.     (>>=)  :: m a -> (a -> m b) -> m b
```

TYPECLASS MONAD

Operador Bind (>>=): Pega um valor **dentro** de um **contexto** (m a), **extrai** o valor **a** (se o contexto permitir), e o **passa** para uma **função a -> m b**, produzindo diretamente **m b**.

A Função return: Envolve um **valor** puro **a** no contexto **mínimo padrão** m a (não confunda com a **palavra-chave return** de linguagens **imperativas**; ela não **interrompe** a execução).

TYPECLASS MONAD

A Notação **do**: É um **açúcar sintático** para chamadas encadeadas de >>=.

TYPECLASS MONAD

```
-- Uso direto de Bind:
computacaoBind = passo1 >>= \x -> passo2 x
>>= \y -> return (x + y)

-- Uso da Notação do:
computacaoDo = do
    x <- passo1
    y <- passo2 x
    return (x + y)
```

MÔNADA MAYBE

A **Mônada Maybe** modela **computações** que podem **falhar** a **qualquer momento**.

```
1.instance Monad Maybe where
2.    return x       = Just x
3.    Nothing >>= _  = Nothing
4.    Just x  >>= f  = f x
```

MÔNADA MAYBE

Comportamento de Curto-Circuito:
Se **qualquer** etapa da sequência avaliar para **Nothing**, o operador >>= **interrompe** a **execução** das etapas subsequentes e **retorna Nothing** imediatamente.

MÔNADA MAYBE

```
divisaoSegura :: Double -> Double -> Maybe Double
divisaoSegura _ 0 = Nothing
divisaoSegura x y = Just (x / y)
```

MÔNADA MAYBE

```
raizSegura :: Double -> Maybe Double
raizSegura x
    | x < 0     = Nothing
    | otherwise = Just (sqrt x)
```

MÔNADA MAYBE

```
-- Encadeamento Monádico
calcular :: Double -> Double -> Maybe Double
calcular x y = do
    q <- divisaoSegura x y
    raizSegura q
```

MÔNADA DE LISTA (NÃO-DETERMINISMO)

Em um contexto **não-determinístico**, em vez de retornar uma **única resposta**, uma função retorna **todas** as **respostas possíveis** em uma **lista**.

MÔNADA DE LISTA (NÃO-DETERMINISMO)

```
instance Monad [] where
    return x = [x]
    xs >>= f = concat (map f xs)
```

Mecânica do Bind:
Para **cada elemento** da lista de **entrada xs**, a função **f** gera uma nova lista. O operador >>= **concatena todos** os resultados em um **único fluxo** plano.

MÔNADA DE LISTA (NÃO-DETERMINISMO)

```
-- Combinações de elementos de duas listas
1. gerarPares :: [a] -> [b] -> [(a, b)]
2. gerarPares xs ys = do
3.     x <- xs
4.     y <- ys
5.     return (x, y)
```

MÔNADAS DE CONTEXTO: READER, WRITER E STATE

Mônada **Reader** r a (Somente Leitura):

- Passa um **ambiente** global **compartilhado r** de forma **implícita** por toda a **cadeia de funções.**
- Primitiva Principal: ask :: Reader r r (**recupera** o **ambiente** de leitura).

Caso de Uso: Leitura de arquivos de configuração, chaves de API ou contexto do sistema.

MÔNADAS DE CONTEXTO: READER, WRITER E STATE

```
import Control.Monad.Reader

-- Estrutura contendo as configurações globais do ambiente
data Configuracao = Configuracao { taxaImposto :: Double }
```

MÔNADAS DE CONTEXTO: READER, WRITER E STATE

```
-- Função que lê a taxa de imposto do ambiente e calcula o preço final

1. calcularPrecoFinal :: Double -> Reader Configuracao Double
2. calcularPrecoFinal precoBase = do
3.     config <- ask
4.     let taxa = taxaImposto config
5.     return (precoBase * (1 + taxa))
```

MÔNADAS DE CONTEXTO: READER, WRITER E STATE

– Simulação de Execução no REPL (GHCi):

```
> let ambiente = Configuracao { taxaImposto
= 0.15 }
> runReader (calcularPrecoFinal 100.0)
ambiente
115.0
```

MÔNADAS DE CONTEXTO: READER, WRITER E STATE

Mônada **Writer w a** (Somente Escrita):

- Produz um valor **a** acompanhado de um **log** ou **acumulador w** (que deve ser um Monoid, como listas ou strings).
- **Primitiva Principal**: tell :: **w** -> **Writer w** () (**acrescenta** um registro ao **log**).

Caso de Uso: Geração de logs de auditoria, rastreamento de passos de algoritmos.

MÔNADAS DE CONTEXTO: READER, WRITER E STATE

```
import Control.Monad.Writer
-- Função que executa cálculos matemáticos e registra o passo a passo no log
processarValor :: Int -> Writer [String] Int
processarValor x = do
    tell ["Iniciando o calculo."]
    tell ["Multiplicando o valor por 2."]
    let dobro = x * 2
    tell ["Somando 10 ao resultado."]
    return (dobro + 10)
```

MÔNADAS DE CONTEXTO: READER, WRITER E STATE

– Simulação de Execução no REPL (GHCi):

```
> runWriter (processarValor 5)
(20, ["Iniciando o calculo.", "Multiplicando o valor por 2.", "Somando 10 ao resultado."])
```

MÔNADA STATE – ESTADO DE LEITURA E ESCRITA

A Mônada **State s a** encapsula funções do tipo **s -> (a, s)**, onde **s** representa o tipo do estado e a é o resultado da computação.

MÔNADA STATE – ESTADO DE LEITURA E ESCRITA

```
-- Disponível em Control.Monad.State
newtype State s a = State { runState :: s -> (a, s) }
```

Primitivas Fundamentais de Control.Monad.State:

- **get** :: State s s — Retorna o estado atual.
- **put** :: s -> State s () — Substitui o estado atual por um novo estado.
- **modify** :: (s -> s) -> State s () — Aplica uma função de transformação ao estado.

MANIPULAÇÃO DE PILHA

```
import Control.Monad.State

type Pilha = [Int]

empilhar :: Int -> State Pilha ()
empilhar x = modify (\st -> x : st)
```

MANIPULAÇÃO DE PILHA

```
desempilhar :: State Pilha Int
desempilhar = do
    st <- get
    case st of
        []     -> error "Pilha vazia"
        (x:xs) -> do
            put xs
            return x
```

MANIPULAÇÃO DE PILHA

```
import Control.Monad.State
-- Função que processa movimentações financeiras atualizando o saldo (estado)
processarTransacoes :: State Int Int
processarTransacoes = do
    modify (+ 100)          -- Deposita 100 no saldo atual
    modify (\saldo -> saldo - 35) -- Saca 35 do saldo atual
    saldoFinal <- get       -- Recupera o estado (saldo) atualizado
    return saldoFinal
```

EXERCICIOS

1- Operações Seguras com Mônada Maybe Implemente a função processarDesconto :: Double -> Double -> Maybe Double que recebe o preço de um produto e um valor de desconto. A função deve retornar Nothing caso o preço seja negativo ou caso o desconto resulte em um preço final menor que zero. Caso contrário, deve retornar Just precoFinal. Utilize a notação do.

EXERCICIOS

2- Produto Cartesiano e Filtragem com Mônada de Lista
Utilizando a notação do sobre a Mônada de Lista [], implemente a função gerarCoordenadasValidas :: Int -> Int -> [(Int, Int)] que gera todos os pares (x, y) tais que x varie de 1 até m e y varie de 1 até n, retendo apenas os pares onde a soma x + y seja um número par.

EXERCICIOS

3- Registro de Auditoria com Mônada Writer
Utilizando o módulo Control.Monad.Writer, implemente a função calculoAuditado :: Int -> Writer [String] Int. A função deve receber um número x, somar 10 a ele emitindo o log "Somado 10", e em seguida multiplicar o resultado por 2 emitindo o log "Multiplicado por 2", retornando o valor final e a lista completa de logs.

EXERCICIOS

4- Manipulação de Estado com Mônada State
Considere um contador simples mantido em estado. Utilizando o módulo Control.Monad.State, implemente a função atualizarContador :: Int -> State Int Int. A função deve somar o valor fornecido ao estado atual (usando modify ou get/put) e retornar o novo valor do contador incrementado.

DÚVIDAS???